

Design Chat and Notification Delivery

Real-time transport is only one part of messaging. A reliable design also needs durable identities, ordering scope, authorization, device synchronization, and explicit receipt semantics. Notifications add preferences, provider limits, retries, and ambiguous delivery outcomes that cannot be solved by a WebSocket alone.

For: Candidates preparing real-time product designs who need to distinguish durable messages, live connections, offline delivery, receipts, and external notification providers.

Suggested pace: 2 weeks of state and failure walkthroughs | Backend and queue fundamentals

What you will be able to do

- Separate durable message storage from transient connection routing.
- Define sent, accepted, delivered, and read states without overclaiming.
- Handle reconnects, duplicate sends, group membership, and offline devices.
- Design notification retries and preferences with bounded cost and backlog.

Build the right mental model

Connections are not the source of truth

A live connection can disappear at any point. Persist a message before acknowledging the promised acceptance boundary, then route it to online devices. Reconnecting clients need a cursor or sequence to recover missed history rather than trusting an in-memory socket buffer.

Order and identity need a scope

A client-generated request ID can deduplicate a retried send, while a server-assigned conversation sequence can establish a display order. Global ordering across every conversation is usually unnecessary. Define how multiple devices and membership changes interact with access.

Receipts describe different events

Provider acceptance, device delivery, and user read acknowledgement are not equivalent. Push notifications are often hints to fetch authoritative state. A repeated provider callback or uncertain timeout needs a state machine, not a new logical message each time.

Study with evidence.

Draw both the durable message path and the online delivery path. For each acknowledgement, write exactly what is known. Use local clients and mocked providers so tests can simulate disconnects and retries without contacting real recipients.

Days 1-3: Message contract

Define conversations, members, message IDs, attachment references, and pagination. Choose per-conversation order and a stable retry identity. Specify edit, delete, and retention semantics before adding real-time transport.

Deliverable and check: An API and data model with clear authorization and acknowledgement boundaries.

Days 4-6: Online and offline paths

Add connection routing and durable catch-up using a cursor. Test reconnect, duplicate send, and a device receiving the same event twice. Keep attachment bytes in a suitable object store rather than every message row.

Deliverable and check: A client that can recover missed history without losing or duplicating visible messages.

Days 7-10: Notifications and groups

Add preferences, quiet hours, group fan-out, provider adapters, and bounded retries. Recheck relevant permissions before delivery and handle removed group members according to the documented history policy.

Deliverable and check: A failure matrix for provider timeout, membership change, and a slow or offline recipient.

Days 11-14: Scale and operate

Estimate concurrent connections, message rate, fan-out, storage retention, and reconnect bursts. Add backpressure, rate limits, and delivery-lag metrics. Discuss regional failure without promising seamless global ordering.

Deliverable and check: A design walkthrough with capacity assumptions and a tested reconnect scenario.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Recover after acknowledgement loss

A sender submits a message, the server stores it, and the connection breaks before the acknowledgement arrives. The sender reconnects and retries while the recipient may already have seen the message.

A worked approach

1. Give the send a stable client operation ID scoped to sender and conversation. Store the ID and the message result atomically, rejecting reuse with a different payload. The retry returns the existing message identity rather than creating another row.
2. Route the durable message to the recipient using the same server message ID. The client deduplicates repeated delivery by that identity and advances a persisted conversation cursor only according to the ordering contract.
3. On reconnect, fetch messages after the last acknowledged cursor and reconcile local pending sends with server results. A missing socket acknowledgement is treated as an unknown outcome, not proof the message was never stored.

Expected result

The logical message remains one message across retries and duplicate transport events. Delivery receipts still represent their specific stage and do not become a claim that the user read it.

Verify the result

- Can a retry with the same ID create a second message?
- Does the recipient deduplicate replayed events?
- Can a removed member fetch history they are no longer authorized to see?

Extend the exercise

Send a push notification for the new message and lose the provider response. Explain why the notification may be duplicated even when the underlying chat message is deduplicated, and how the UI can tolerate that.

Make the learning stick.

Why use WebSockets instead of polling?

They can reduce repeated request overhead and support low-latency bidirectional delivery, but require connection management and reconnect handling. Polling or server-sent events may be appropriate for simpler requirements; choose from the actual interaction pattern.

Does message ordering require one global sequence?

Usually a conversation or another entity scope is sufficient. A global sequence creates unnecessary coordination unless a requirement truly depends on total order across unrelated conversations.

Can push delivery be guaranteed exactly once?

Not generally across external providers and devices. Define accepted and delivered states, use stable identities where supported, and make the application tolerate duplicate hints while authoritative message state remains correct.

Avoid these traps

- Treating live connection state as durable message history.
- Equating provider acceptance with user read acknowledgement.
- Ignoring reconnect storms, duplicate events, and membership changes.

Ready for the next step?

- Message identity and ordering scope are explicit.
- Offline catch-up and retries preserve user-visible correctness.
- Authorization applies to history and attachments.
- Receipts and provider failure semantics are honestly described.

Reliable next reads

- [MDN WebSocket API](#)
- [Azure event-driven architecture](#)
- [Azure queue-based load leveling](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.